

MANUAL TÉCNICO

Quetzal Space Defender - Side Scroller

Curso: Introducción a la Programación y Computación 1

Práctica: Práctica 2

Contenido

1. Descripción general
2. Requisitos del sistema
3. Estructura del proyecto
4. Descripción de clases
5. Uso de hilos
6. Mecánicas del juego
7. Sistema de colisiones
8. Persistencia con vectores
9. Gráfica de mejores puntajes
10. Generación de reportes
11. Compilación y ejecución
12. Dependencia externa
13. Pruebas realizadas
14. Evidencias sugeridas
15. Conclusiones

Descripción general

Quetzal Space Defender es un videojuego tipo **Side Scroller** desarrollado en Java. El jugador controla una nave que se desplaza dentro de un escenario espacial mientras aparecen enemigos y objetos especiales desde el lado derecho de la pantalla.

El sistema utiliza programación orientada a objetos, estructuras de tipo **Vector**, interfaz gráfica con Swing e hilos para controlar de forma independiente el movimiento de enemigos y proyectiles.

El proyecto permite:

- Crear pilotos.
- Seleccionar una dificultad y un tipo de nave.
- Iniciar partidas.
- Mover la nave del jugador.
- Disparar proyectiles.
- Generar enemigos y objetos especiales.
- Detectar colisiones.
- Registrar puntajes.
- Consultar el historial de partidas.
- Mostrar un top de puntajes mediante JFreeChart.
- Exportar un reporte en formato HTML.

Requisitos del sistema

Para ejecutar el proyecto se recomienda contar con:

- Java JDK 17 o superior.
- Maven.
- Visual Studio Code, NetBeans o IntelliJ IDEA.
- Conexión a Internet la primera vez que Maven descargue la dependencia de JFreeChart.

Estructura del proyecto

```
QuetzalSpaceDefender/
├── pom.xml
├── README.md
├── docs/
│   ├── MANUAL_TECNICO.md
│   ├── MANUAL_USUARIO.md
│   └── DIAGRAMA_FLUJO.md
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── quetzal/
│   │   │   │   │   ├── spacedefender/
│   │   │   │   │   │   ├── Main.java
│   │   │   │   │   │   ├── AppData.java
│   │   │   │   │   │   ├── Pilot.java
│   │   │   │   │   │   ├── Difficulty.java
│   │   │   │   │   │   ├── GameConfig.java
│   │   │   │   │   │   ├── GameRecord.java
│   │   │   │   │   │   ├── MainMenuFrame.java
│   │   │   │   │   │   ├── PilotFrame.java
│   │   │   │   │   │   ├── GameFrame.java
│   │   │   │   │   │   ├── GamePanel.java
│   │   │   │   │   │   ├── GameObjectType.java
│   │   │   │   │   │   ├── MovingObject.java
│   │   │   │   │   │   ├── Projectile.java
│   │   │   │   │   │   ├── HistoryFrame.java
│   │   │   │   │   │   ├── TopScoresFrame.java
│   │   │   │   │   │   └── ReportExporter.java
│   │   └── resources/
```

Descripción de clases

Main.java

Es el punto de entrada del programa. Utiliza `SwingUtilities.invokeLater()` para iniciar la interfaz gráfica de forma segura y crea una instancia de `AppData`, la cual es compartida por las diferentes ventanas del sistema.

Responsabilidades principales:

- Iniciar la aplicación.
- Crear los datos globales.
- Mostrar el menú principal.

AppData.java

Administra la información que se mantiene durante la ejecución del programa.

Utiliza estructuras **Vector** para almacenar:

- Pilotos registrados.
- Historial de partidas.

También contiene métodos para:

- Agregar pilotos.
- Evitar nombres duplicados.
- Registrar resultados de partidas.
- Obtener copias del historial.
- Ordenar los resultados por puntaje.

Pilot.java

Representa a un piloto del juego.

Atributos principales:

- Nombre del piloto.
- Dificultad seleccionada.

Cada piloto queda asociado a un tipo de nave por medio de la dificultad.

Difficulty.java

Es un **enum** que representa las tres dificultades disponibles.

Dificultad	Nave	Velocidad	Tiempo entre disparos
Fácil	Explorador	Alta	2000 ms
Normal	Caza Estelar	Media	1000 ms
Difícil	Acorazado	Baja	300 ms

La clase almacena los valores utilizados por el juego para determinar la velocidad del jugador y el tiempo mínimo entre disparos.

GameConfig.java

Contiene valores de configuración generales del juego. Esta clase permite mantener constantes importantes separadas de la lógica principal.

Ejemplos de valores configurables:

- Duración de la partida.
- Cantidad inicial de vidas.
- Tamaño de la ventana.
- Valores de puntaje.

GameRecord.java

Representa una partida terminada.

Almacena:

- Nombre del piloto.
- Dificultad.
- Puntaje obtenido.
- Fecha y hora de la partida.

Los registros son utilizados en el historial y en el top de puntajes.

MainMenuFrame.java

Representa la ventana principal del programa.

Contiene las opciones:

- Jugar.
- Crear Piloto.
- Top de Puntajes.
- Historial.
- Salir.

Desde esta ventana se navega hacia las demás funciones del sistema.

PilotFrame.java

Permite crear nuevos pilotos.

Realiza las siguientes validaciones:

- El nombre no puede estar vacío.
- El nombre debe tener una longitud mínima.
- No se permiten pilotos duplicados.

También permite seleccionar una de las tres dificultades disponibles.

GameFrame.java

Es la ventana que contiene el juego.

Su responsabilidad principal es crear y mostrar un objeto `GamePanel`, donde se ejecuta toda la lógica de la partida.

GamePanel.java

Es la clase principal del videojuego.

Se encarga de:

- Dibujar el escenario.
- Dibujar la nave.
- Leer las teclas del jugador.
- Crear enemigos y objetos especiales.
- Crear proyectiles.
- Ejecutar hilos.
- Detectar colisiones.
- Actualizar puntajes.
- Controlar vidas.
- Controlar el tiempo de la partida.
- Finalizar la partida.

La clase implementa `KeyListener` para detectar el teclado.

Controles:

Acción	Tecla
Arriba	W o flecha arriba
Abajo	S o flecha abajo
Izquierda	A o flecha izquierda
Derecha	D o flecha derecha
Disparar	Espacio
Terminar partida	ESC

GameObjectType.java

Es un `enum` que identifica los tipos de objetos que pueden aparecer durante la partida.

Tipos disponibles:

- `ENEMY`
- `SNITCH`
- `BLUDGER`
- `QUAFFLE`

MovingObject.java

Representa los objetos que se desplazan desde el borde derecho hacia el borde izquierdo.

Almacena información como:

- Posición X.
- Posición Y.
- Ancho.
- Alto.
- Tipo de objeto.
- Estado activo o inactivo.

Esta clase es utilizada para enemigos y objetos especiales.

Projectile.java

Representa un proyectil disparado por el jugador.

Cada proyectil posee:

- Posición X.
- Posición Y.
- Estado activo.
- Área de colisión.

Cada proyectil se desplaza de izquierda a derecha utilizando un hilo independiente.

HistoryFrame.java

Muestra una tabla con el historial de partidas.

La tabla contiene:

- Fecha.
- Piloto.
- Dificultad.
- Puntaje.

También contiene un botón para exportar el reporte en formato HTML.

TopScoresFrame.java

Muestra los mejores puntajes utilizando **JFreeChart**.

Los registros son ordenados de mayor a menor y se muestran en una gráfica de barras.

ReportExporter.java

Genera el reporte del historial y del top de puntajes.

Utiliza:

- `FileWriter`
- `PrintWriter`

El archivo generado utiliza HTML y CSS, lo que permite visualizarlo en un navegador y posteriormente guardarlo como PDF si se requiere.

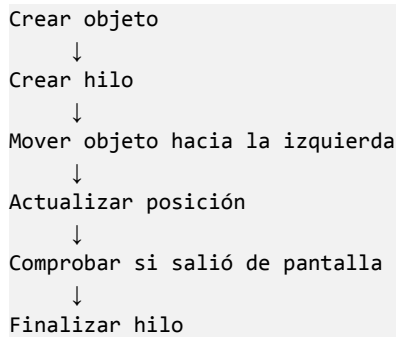
Uso de hilos

Los hilos son una de las partes principales del proyecto.

Hilo de enemigos y objetos

Cada enemigo u objeto que aparece en pantalla posee su propio hilo. Dicho hilo actualiza continuamente su posición en el eje X para simular el movimiento horizontal de derecha a izquierda.

Flujo general:



Hilo de proyectiles

Cada proyectil disparado genera un hilo independiente.

El hilo:

1. Mueve el proyectil hacia la derecha.
2. Verifica si existe una colisión.
3. Actualiza el puntaje cuando corresponde.
4. Se detiene cuando el proyectil impacta o sale de la pantalla.

Hilo del Bludger

Cuando se activa el efecto de un Bludger, el movimiento del jugador queda bloqueado durante aproximadamente dos segundos.

La pausa se controla mediante un hilo separado para evitar detener completamente la interfaz gráfica.

Mecánicas del juego

Enemigo

Los enemigos están identificados con la letra **E**.

Si un enemigo colisiona con la nave del jugador, se reduce la cantidad de vidas disponibles.

Snitch espacial

El Snitch está identificado con la letra **S**.

Efectos:

- Otorga 150 puntos.
- Destruye los enemigos visibles.

Bludger

El Bludger está identificado con la letra **B**.

Efecto:

- Bloquea temporalmente el movimiento del jugador durante aproximadamente dos segundos.

Quaffle

El Quaffle está identificado con la letra **Q**.

Efecto:

- Suma 10 puntos al puntaje actual.

Sistema de colisiones

Para detectar colisiones se utilizan objetos **Rectangle**.

Cada elemento posee un área rectangular determinada por:

- Posición X.
- Posición Y.
- Ancho.
- Alto.

La detección se realiza mediante el método:

```
intersects()
```

Se verifican principalmente dos tipos de colisión:

1. Proyectil contra objeto.
2. Nave del jugador contra objeto.

Dependiendo del tipo de objeto se ejecuta la acción correspondiente.

Persistencia con vectores

La información del sistema se almacena durante la ejecución mediante la clase **Vector**.

Se utilizan principalmente:

```
Vector<Pilot>
```

para pilotos y:

```
Vector<GameRecord>
```

para el historial de partidas.

El uso de **Vector** permite agregar, recorrer, ordenar y consultar los datos durante la ejecución del sistema.

Gráfica de mejores puntajes

Para mostrar los mejores resultados se utiliza la librería **JFreeChart**.

Los pasos realizados son:

1. Obtener una copia del historial.
2. Ordenar los registros de mayor a menor.
3. Seleccionar los mejores resultados.
4. Crear un **DefaultCategoryDataset**.
5. Crear una gráfica de barras.
6. Mostrarla mediante un **ChartPanel**.

Generación de reportes

El sistema puede generar un reporte HTML con:

- Historial completo.
- Piloto.
- Dificultad.
- Fecha.
- Puntaje.
- Top de mejores resultados.

La generación se realiza exclusivamente mediante clases relacionadas con `java.io`, principalmente:

```
FileWriter  
PrintWriter
```

El resultado es un archivo `.html` que puede abrirse con cualquier navegador moderno.

Compilación y ejecución

Desde la terminal, ubicándose en la carpeta donde se encuentra `pom.xml`, ejecutar:

```
mvn clean compile
```

Si la compilación es correcta aparecerá:

```
BUILD SUCCESS
```

Posteriormente ejecutar:

```
mvn exec:java
```

También puede ejecutarse `Main.java` directamente desde un IDE configurado para proyectos Maven.

Dependencia externa

El proyecto utiliza JFreeChart para las gráficas.

La dependencia se encuentra declarada en `pom.xml`:

```
<dependency>  
  <groupId>org.jfree</groupId>  
  <artifactId>jfreechart</artifactId>  
  <version>1.5.4</version>  
</dependency>
```

Maven se encarga de descargar e incorporar la librería al proyecto.

Pruebas realizadas

Durante las pruebas del sistema se comprobó:

- Creación de pilotos.
- Validación de nombres duplicados.
- Selección de las tres dificultades.
- Inicio y finalización de partidas.
- Movimiento de la nave.
- Generación de enemigos.
- Generación de proyectiles.
- Uso independiente de hilos.
- Colisiones.
- Aparición de Quaffle.
- Aparición y efecto del Bludger.
- Aparición del Snitch.
- Registro del historial.
- Top de puntajes.
- Generación de gráfica con JFreeChart.
- Exportación del reporte HTML.

Evidencias sugeridas

Para complementar este manual se pueden agregar capturas de:

1. Menú principal.
2. Creación de piloto.
3. Selección de dificultad.
4. Partida en ejecución.
5. Quaffle en pantalla.
6. Mensaje de Bludger bloqueando el movimiento.
7. Snitch en pantalla.
8. Historial de partidas.
9. Gráfica de mejores puntajes.
10. Reporte HTML abierto en el navegador.

Conclusiones

El desarrollo de Quetzal Space Defender permitió aplicar conceptos fundamentales de programación en Java, principalmente programación orientada a objetos, manejo de estructuras de datos, interfaces gráficas y programación concurrente mediante hilos.

La separación del sistema en distintas clases facilita la lectura y mantenimiento del código. Cada clase posee una responsabilidad específica, lo cual permite modificar componentes del programa sin afectar innecesariamente el resto de la aplicación.

El uso de hilos permite que enemigos y proyectiles se desplacen de manera independiente, mientras que el uso de vectores permite mantener el historial de pilotos y partidas durante la ejecución.

Finalmente, JFreeChart y la generación de reportes HTML permiten presentar los resultados de una forma gráfica y organizada.